

Big Data Frameworks:**1.Data Collection:**

This is the step where data is gathered from different places like sensors, websites, apps, machines or social media. The goal is to bring all the incoming data into one system so it can be used later.

2.Data Storage: After collecting the data, it needs to be stored safely. Big data systems can store huge amounts of information across many computers. They keep the data organized and make sure it can be accessed whenever needed.

3.Data Preprocessing: Raw data is usually messy. Preprocessing cleans the data by removing errors, filling missing values and converting it into a proper format. This step makes the data ready for analysis.

4.Data Analysis: In this step, tools and algorithms study the cleaned data to find patterns, trends or useful information. This helps answer important questions or solve problems using the data.

5.Visualization and Decision Making: The results of the analysis are shown using charts, graphs or dashboards so they are easy to understand. Based on these visuals, organizations can make better decisions.

Big Data Management:

1.Data Collection: This is the step where data is gathered from many places. These sources can be sensors, mobile apps, websites, machines, CCTV cameras, social media or business software. The goal is to bring all this data into one system so it can be stored and processed. Because data comes in large amounts and at high speed, special tools are used to handle it smoothly.

2.Data Storage: After collecting data, it must be kept somewhere safe and organized. Big data storage systems can hold massive amounts of information.

This storage can be:

- Cloud storage (like S3 or Google Cloud Storage)
- Distributed systems (like HDFS)
- Databases (like MongoDB or Cassandra)

Storage systems also make sure data is not lost if a machine fails.

3.Data Processing: Raw data is usually messy and cannot be directly used.

Processing helps clean and prepare the data by:

- Removing duplicate or wrong values
- Filling missing information
- Changing formats (like converting text to numbers)
- Combining data from different sources

This step makes the data reliable and ready for analysis.

4.Data Governance: Data governance sets rules for how data should be used.

It explains:

- Who can access the data
- How long the data should be stored
- What quality standards should be maintained

It helps keep data organized, trustworthy and compliant with laws.

5.Data Integration: Data often comes from different systems and in different formats.

Integration brings all this data together so it looks uniform and works well in analysis.

For example, combining data from a website, a mobile app and a sensor into one dataset.

Big Analytics:

1.Descriptive Analytics: Descriptive analytics focuses on summarizing and interpreting historical data to understand what has already occurred. It uses techniques such as data aggregation, data mining and basic statistical analysis to present past trends and patterns.

Ex: Generating a report showing last year's total sales.

2.Diagnostic Analytics: Diagnostic analytics examines data more deeply to determine the causes behind past events. It uses techniques like drill-down analysis, data discovery and correlation analysis to identify why something happened.

Ex: Finding that sales decreased because customer complaints increased.

3.Predictive Analytics: Predictive analytics uses statistical models, machine learning and historical data to estimate future outcomes. It identifies patterns in past data and applies them to forecast what is likely to happen next.

Ex: Predicting next month's sales based on past sales trends.

4.Prescriptive Analytics: Prescriptive analytics recommends the best possible actions by analyzing predictions, constraints and possible outcomes. It uses optimization models, simulation and decision rules to suggest what should be done to achieve desired goals.

Ex: Suggesting the optimal price of a product to maximize profit.

5.Real-Time Analytics: Real-time analytics processes and analyzes data the moment it is generated. It relies on streaming platforms and in-memory computing to deliver instant insights for time-sensitive decisions.

Ex: Detecting fraudulent credit-card transactions while they are happening.

5 V 's:

1.Volume: Volume refers to the huge amount of data generated every second from sources like social media, sensors, machines, transactions and devices. Big data systems must be able to store, process and manage datasets that are too large for traditional systems.

2.Velocity: Velocity is the speed at which data is created, collected and processed. Many applications require data to be handled in real time or near real time. High velocity data often comes from streaming sources like IoT devices, online transactions and live feeds.

3.Variety: Variety represents the different forms of data. Big data is not only structured (tables and spreadsheets) but also semi-structured (JSON, XML) and unstructured (videos, images, audio, emails). Managing and integrating all these formats is a key challenge.

4.Veracity: Veracity refers to the quality, accuracy and trustworthiness of the data. Big data often contains inconsistencies, noise or missing values. Ensuring reliable data is important for producing correct insights and decisions.

5.Value: Value is the usefulness of the data. Collecting massive data is pointless unless it provides meaningful insights or benefits. Value focuses on converting raw data into knowledge that supports decision-making, efficiency and innovation.

Current Challenges and Trends in Big Data:

Current Challenges in Big Data:

- 1.Data Quality:** Big data often contains errors, duplicates, missing values or noise. Ensuring accuracy and reliability is difficult because data comes from many different sources.
- 2.Data Security and Privacy:** Protecting sensitive data is a major challenge. Large datasets increase the risk of data breaches, unauthorized access and misuse. Maintaining privacy regulations is also becoming more complex.
- 3.Storage and Scalability:** As data continues to grow, organizations struggle to store it efficiently. Systems must scale quickly without losing performance or becoming too expensive.
- 4.Data Integration:** Data comes in different formats such as text, images, videos and sensor readings. Combining all these forms into a single usable dataset is challenging.
- 5.Processing Speed:** Analyzing huge datasets requires high computing power. Traditional systems cannot handle the speed needed for real-time or near real-time processing.
- 6.Skilled Workforce:** There is a shortage of professionals skilled in big data tools, analytics, machine learning and cloud technologies. This slows down big data projects.
- 7.High Cost of Tools and Infrastructure:** Big data platforms, storage systems and cloud services can be expensive. Many organizations struggle to balance cost with performance.

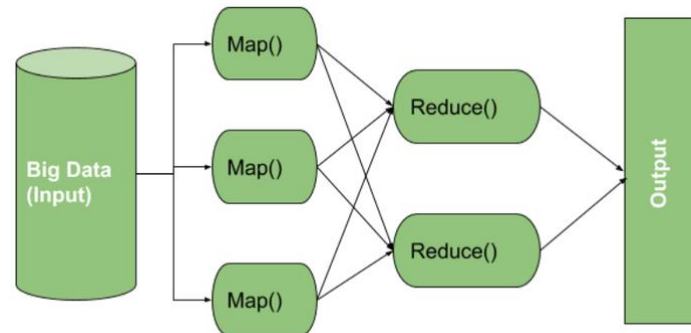
Current Trends in Big Data:

- 1.Cloud-Based Big Data Platforms:** More companies are moving their data to cloud services. Cloud-based tools make storage, processing and analysis faster, cheaper and more scalable.
- 2.Real-Time Analytics:** Organizations now want insights instantly. Real-time data processing is becoming common in areas like finance, IoT, healthcare and transportation.
- 3.Artificial Intelligence and Machine Learning Integration:** AI and ML are being used to automate data analysis, make predictions and improve decision-making. Big data and AI now work together closely.
- 4.Edge Computing:** Instead of sending all data to the cloud, some processing happens at the device or sensor level. This reduces delay and improves real-time applications.
- 5.Data Lakes:** Companies are using data lakes to store raw data in its original form. This helps in flexible analysis and advanced analytics.
- 6.Automation in Data Management:** Tools are emerging that automatically clean, classify and organize data. This reduces manual work and improves data quality.
- 7.Increased Focus on Data Governance:** Organizations are adopting strict rules for data usage. This includes policies for security, privacy, ownership and compliance with laws.

MapReduce Algorithm: MapReduce is the processing engine of Hadoop. While HDFS is responsible for storing massive amounts of data, MapReduce handles the actual computation and analysis. It provides a simple yet powerful programming model that allows developers to process large datasets in a distributed and parallel manner. It is a two-phase data processing model in Hadoop:

Map Phase: Breaks the data into smaller chunks, processes them, and generates intermediate (key, value) pairs.

Reduce Phase: Aggregates and merges the intermediate results to produce the final output.



1. Input Phase: In the input phase, the original data is divided into smaller blocks. These blocks are stored in HDFS and are given to different machines to process. Each block becomes the input for a mapper. This step allows large datasets to be processed in parallel.

2. Mapper Phase: In this phase, each mapper reads its input block and processes the data. The mapper converts the raw data into key–value pairs. This helps organize the information and prepares it for grouping. The output of the mapper is a list of intermediate key–value pairs.

3. Shuffle and Sort Phase: This is the intermediate phase between map and reduce. In this stage, all key–value pairs from the mappers are collected, grouped by key and sorted. The system ensures that all values related to the same key are sent to the same reducer. This step organizes the data so that the reducer can work efficiently.

4. Reducer Phase: In the reducer phase, each reducer receives a key and all the values associated with it. The reducer combines or processes these values to produce the final result for that key. Common operations include adding, counting, averaging or merging.

5. Output Phase: In the final phase, the reducer's results are written back to HDFS. The output is stored as files that can be used for further analysis or applications. This completes the MapReduce job.

HDFS: HDFS is a distributed file system designed to store very large files across many machines. It provides high reliability, fault tolerance and fast data access by splitting data into blocks and storing them across a cluster.

Working:

1. Data is Split into Blocks: When a file is uploaded to HDFS, it is not stored as a single file. Instead, it is divided into fixed-size blocks (default: 128 MB or 64 MB).

Ex: A 300 MB file → split into 3 blocks.

Splitting helps handle big files easily and allows parallel processing.

2. Blocks are Distributed Across DataNodes: HDFS has two main components:

- NameNode: the master
- DataNodes: the workers

The blocks created from the file are stored on different DataNodes in the cluster.

This distribution allows Hadoop to read and process files faster.

3.Replication for Fault Tolerance: Every block is stored multiple times on different DataNodes. This is called the replication factor (default: 3).

Ex: One block is stored on three different nodes. So even if one node fails, the data is safe. Replication ensures reliability and high availability.

4.NameNode Manages Metadata: The NameNode does not store the data itself. It stores information about:

- Which blocks make up a file
- Where each block is stored
- Replication status
- File permissions

Metadata is stored in memory for quick access.

5.Client Requests: When a user reads or writes a file:

- The client contacts the NameNode
- The NameNode tells the client which DataNodes hold the required blocks
- The client then directly reads from or writes to those DataNodes

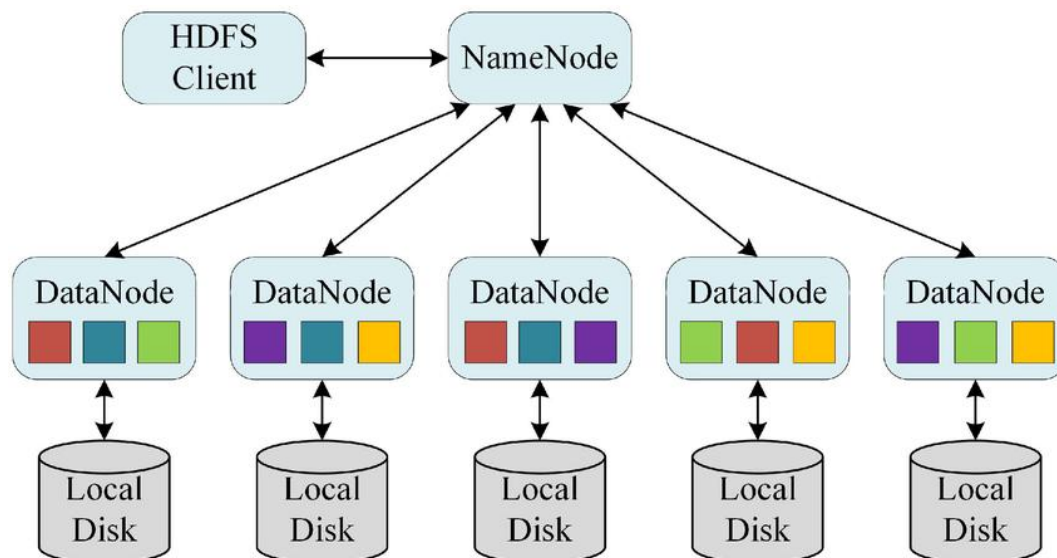
The NameNode only guides; it doesn't handle the actual data.

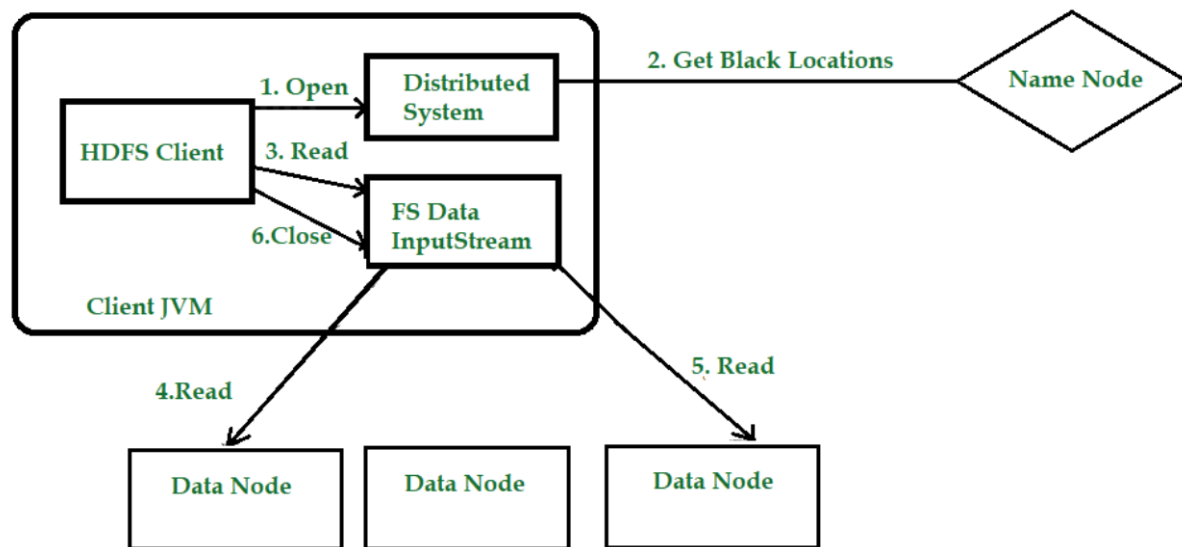
6.DataNode Responsibilities: Each DataNode:

- Stores data blocks
- Sends heartbeat signals to the NameNode
- Reports block information
- Manages block read/write requests

If a DataNode stops sending heartbeats, the NameNode marks it as "dead" and replicates its blocks elsewhere.

7.Parallel Processing: Since blocks are stored across many nodes, Hadoop can process them in parallel, which makes data analysis very fast. This is the foundation of MapReduce and other big data tools.



Read Anatomy of file in Hadoop:

1.The client opens the file it wishes to read by calling open() on the File System Object(which for HDFS is an instance of Distributed File System).

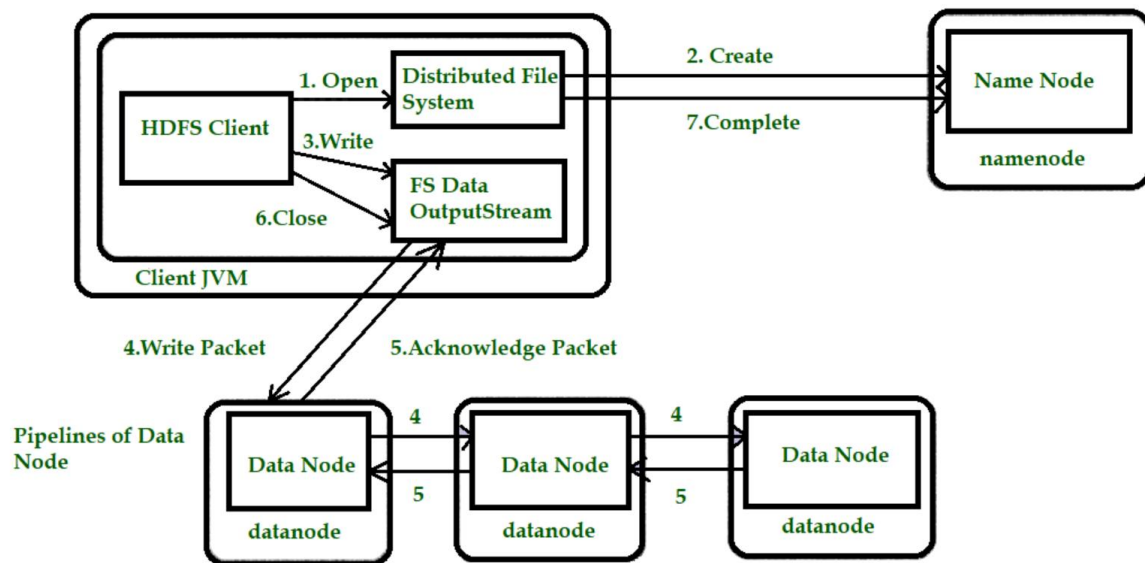
2.DFS contacts the NameNode to find out which DataNodes hold the first blocks of the file. The NameNode returns the block locations. DFS then gives the client an FSDataInputStream, which uses DFSInputStream to handle reading data from those DataNodes.

3.The client then calls read() on the stream. DFSInputStream, which has stored the info node addresses for the primary few blocks within the file, then connects to the primary (closest) data node for the primary block in the file.

4.Data is streamed from the data node back to the client, which calls read() repeatedly on the stream.

5.When a block finishes, DFSInputStream closes the connection and connects to the best DataNode for the next block. This switch happens automatically, so the client still feels like it's reading one continuous stream. As needed, DFSInputStream also asks the NameNode for the locations of upcoming blocks.

6.When the client has finished reading the file, a function is called, close() on the FSDataInputStream.

Write Anatomy of file in Hadoop:

1.The client creates the file by calling create() on DistributedFileSystem(DFS).

2.NameNode creates the file: The client requests to create a new file. DFS sends an RPC to the NameNode. The NameNode checks:

- If the file already exists
- If the client has permission
- If everything is correct, the NameNode creates an empty file entry (no blocks yet).
- If not, it throws an IOException.
- The client then receives an FSDataOutputStream to start writing data.

3.Client starts writing data: As the client writes, the DFSOutputStream breaks the data into small packets. These packets go into a queue called the data queue. A component called DataStreamer takes packets from the queue and asks the NameNode to allocate blocks. The NameNode selects a set of DataNodes (usually 3 for replication). These DataNodes form a pipeline.

How the pipeline works:

- The client sends packets to the **first DataNode**.
- The first DataNode stores the packet and forwards it to the **second DataNode**.
- The second DataNode stores it and forwards it to the **third DataNode**.

This way, all replicas are stored as the data flows through the pipeline.

4.Similarly, the second data node stores the packet and forwards it to the third (and last) data node in the pipeline.

5.The DFSOutputStream sustains an internal queue of packets that are waiting to be acknowledged by data nodes, called an "ack queue".

6.This action sends up all the remaining packets to the data node pipeline and waits for acknowledgments before connecting to the name node to signal whether the file is complete or not.

NameNode, Secondary NameNode and DataNode:

1.NameNode: The NameNode is the central component of the HDFS file system. It manages the file system namespace, which includes the entire file system tree and all metadata about files and directories.

Metadata Stored by the NameNode: For every file, the NameNode stores metadata such as File name, File path, Number of blocks, Block IDs, Replication level. This metadata is stored on the local disk of the NameNode.

Files Used to Store Metadata: The NameNode uses two important files to store this information, FsImage and EditLog.

Block Location Information:

- The NameNode also keeps the locations of DataNodes that store each block of every file.
- Using this information, the NameNode can reconstruct any file by identifying where all its blocks are stored.

Client Interaction:

- Clients must communicate with the NameNode to add, copy, move or delete a file in HDFS.
- Since block information is maintained by the NameNode, any client that wants to read a file must first get its BlockReport from the NameNode.
- The NameNode then returns a list of DataNodes where the blocks of that file are stored.

2.Secondary NameNode: The Secondary NameNode is a helper node for the NameNode. It is not a backup NameNode and cannot immediately replace the NameNode in case of failure.

The two key files used by the NameNode:**1.EditLog:**

- Stores all file system write operations performed by clients.
- Every change (add, delete, modify) is recorded here.

2.FSImage:

- Contains a complete snapshot of the file system metadata at the time the NameNode last started.
- After startup, all new changes go into the EditLog.

Why Secondary NameNode is Needed:

- When the NameNode restarts, it loads metadata from FSImage and then applies all operations from the EditLog.
- Because NameNode restarts rarely, the EditLog can grow very large.
- Merging a huge EditLog with FSImage at startup takes a long time, keeping the entire HDFS offline.

This is inefficient, so a helper is needed to regularly merge these files and keep the metadata up-to-date.

Role of Secondary NameNode: The main job of the Secondary NameNode is to periodically checkpoint the NameNode's metadata. This means it merges FSImage and EditLog to create a new, updated FSImage. This reduces the size of the EditLog and speeds up NameNode recovery.

Checkpointing Process (Steps):

- Secondary NameNode downloads the latest FSImage and EditLog from the NameNode.
- It applies all transactions from the EditLog to the FSImage, creating a new merged FSImage.
- The updated FSImage is then sent back to the NameNode.

This ensures the NameNode always has a recent and clean version of metadata.

Configuration Parameters for Checkpointing: These parameters control how often checkpointing happens (set in *hdfs-site.xml*),

1.dfs.namenode.checkpoint.period:

- Maximum time gap between two checkpoints.
- Default: 1 hour

2.dfs.namenode.checkpoint.txns

- Maximum number of uncheckpointed transactions allowed before forcing an immediate checkpoint.:
- Default: 1 million

3.DataNode:

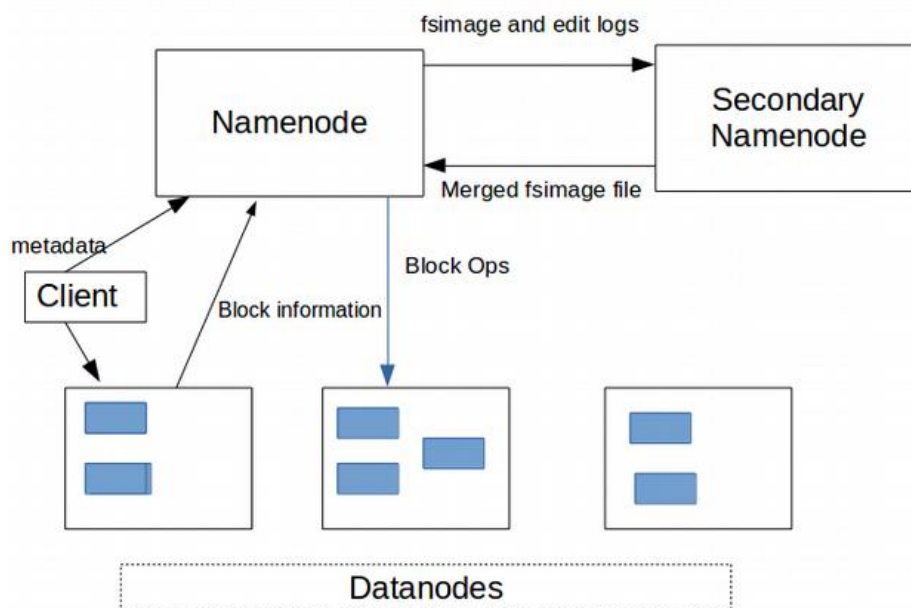
- i. In an HDFS cluster, the actual data blocks of files are stored on multiple DataNodes.
- ii. When a client wants to access a file, it first gets the list of DataNodes storing the file's blocks from the NameNode.
- iii. After that, the DataNodes handle the read and write requests directly from the client.
- iv. The actual user data never passes through the NameNode; it only provides block locations.

Functions of DataNodes:

They are responsible for, Storing data blocks, Deleting blocks when instructed by the NameNode, Replicating blocks to maintain the required replication factor

Block Reports:

- Each DataNode periodically sends a BlockReport to the NameNode.
- A BlockReport contains a complete list of all blocks stored on that DataNode.
- These reports help the NameNode keep accurate and updated metadata about block locations.



Common Shell Command of HDFS:**1. Check HDFS Directories:**

- **hdfs dfs -ls /path:** Shows the files and folders inside the given directory.
- **hdfs dfs -ls -R /path:** Shows everything inside the directory, including subfolders.

2. Create Directories:

- **hdfs dfs -mkdir /path:** Creates a new directory.
- **hdfs dfs -mkdir -p /parent/child:** Creates parent folders if they don't exist.

3. Upload Files to HDFS:

- **hdfs dfs -put localfile /hdfs/path:** Copies a file from your local system to HDFS.
- **hdfs dfs -copyFromLocal file /hdfs/path:** Same as put.

4. Download Files from HDFS:

- **hdfs dfs -get /hdfs/file localpath:** Copies a file from HDFS to your local system.
- **hdfs dfs -copyToLocal /hdfs/file localpath:** Same as get.

5. View and Read Files:

- **hdfs dfs -cat /path/file:** Displays the content of the file.
- **hdfs dfs -tail /path/file:** Shows the last few lines of the file.
- **hdfs dfs -head /path/file:** Shows the first few lines.

6. Move and Copy Files Within HDFS:

- **hdfs dfs -mv /source /dest:** Moves a file or folder inside HDFS.
- **hdfs dfs -cp /source /dest:** Copies a file or folder inside HDFS.

7. Delete Files and Directories:

- **hdfs dfs -rm /path/file:** Deletes a file.
- **hdfs dfs -rm -r /path:** Deletes a directory and everything inside it.

8. Check File Status:

- **hdfs dfs -stat /path/file:** Shows file details like size, modification time, etc.

9. Disk Usage:

- **hdfs dfs -du /path:** Shows size of files inside a folder.
- **hdfs dfs -df /path:** Shows free and used space of the HDFS system.

10. Permissions:

- **hdfs dfs -chmod 755 /path:** Changes file or folder permissions.
- **hdfs dfs -chown user:group /path:** Changes owner and group.
- **hdfs dfs -chgrp group /path:** Changes group only.

11. Create Empty File:

- **hdfs dfs -touchz /path/file:** Creates an empty file in HDFS.

12. Test Conditions:

- **hdfs dfs -test -e /path:** Checks if a file exists.
- **hdfs dfs -test -d /path:** Checks if it is a directory.
- **hdfs dfs -test -z /path:** Checks if file is zero bytes.

13. File Merge:

- **hdfs dfs -getmerge /hdfs/dir/ localfile:** Merges multiple small files into one file on local system.

14. Report HDFS Summary:

- **hdfs dfsadmin -report:** Shows total capacity, used space, number of datanodes and more.

Apache Pig:

1. Pig Represents Big Data as data flows. Pig is a high-level platform or tool which is used to process the large datasets. It provides a high-level of abstraction for processing over the MapReduce.

2. It provides a high-level scripting language, known as *Pig Latin* which is used to develop the data analysis codes.

3. First, to process the data which is stored in the HDFS, the programmers will write the scripts using the Pig Latin Language. Internally *Pig Engine* (a component of Apache Pig) converted all these scripts into a specific map and reduce task. But these are not visible to the programmers in order to provide a high-level of abstraction.

4. Pig Latin and Pig Engine are the two main components of the Apache Pig tool. The result of Pig always stored in the HDFS.

5. Note: Pig Engine has two type of execution environment i.e. a *local execution environment* in a single JVM (used when dataset is small in size) and *distributed execution environment* in a Hadoop Cluster.

Common Pig Command:**1. Common Pig Shell Commands:**

- **Start Pig in Local Mode:** `pig -x local`
- **Start Pig in MapReduce (Hadoop) Mode:** `pig`

2. Basic Shell Commands Inside Pig:

- **Clear the Screen:** `clear`
- **Show Help:** `help`
- **Quit the Pig Shell:** `quit` or `exit`

3. File and Script Commands:

- **Run a Pig Script File:** `run myscript.pig`
- **Execute a Script and Return to Shell:** `exec myscript.pig`
- **See Current Working Directory:** `pwd`
- **Change HDFS Directory:** `cd /user/hadoop`
- **List Files in HDFS Directory:** `ls /path` or `fs -ls /`

4.Pig script and command:

- **Load Data:** A = LOAD 'file.txt' USING PigStorage(',') AS (field1:int, field2:chararray);
- **Store Data:** STORE A INTO 'output_folder' USING PigStorage(',');
- **Describe a Relation:** DESCRIBE A;
- **Show Schema:** DUMP A;
- **Filter Data:** B = FILTER A BY field1 > 10;
- **Projection (Select Columns):** C = FOREACH A GENERATE field1, field2;
- **Order / Sort:** S = ORDER A BY field1 ASC;
- **Join:** A = JOIN data By id, marks By id;
- **nano s.csv:** Opens the s.csv file in the nano editor for creating or editing.
- **nano local load_data.pig:** Opens the Pig script file in nano so you can write or modify the script.
- **pig -x local load_data.pig:** Runs the load_data.pig script in local mode on your machine.

Hive: